CareCompass - Reflection

The most critical bug I found and fixed was that favorites existed in the API but not in the user interface. Users could save a resource from the detail page, yet there was no reliable place to review or remove saved items afterward. That broke an important part of the MVP user flow. I fixed it by adding a Favorites page, wiring it into navigation for signed-in users, and confirming save/list/remove behavior through both manual checks and automated API tests.

The most important security issue resolved was weak request and authentication hardening. Before this pass, the app relied mainly on basic JWT checks and prepared SQL statements, but it lacked strong rate limiting on login/register, Helmet security headers, stricter CORS handling, tighter JSON body limits, and consistent sanitization of user-supplied text. Those gaps increase brute-force and injection risk if the app were exposed beyond a local workshop environment. I addressed them by adding security middleware, sanitizing auth/AI/admin inputs, strengthening password rules for new accounts, using a stronger JWT secret pattern, and verifying that injection-style and script-like inputs no longer crash the system or return unsafe markup.

AI helped significantly with testing and security. Instead of manually inventing every edge case, I asked Cursor to inventory features and buttons, propose a full security audit checklist, implement fixes across frontend and backend files, and generate an automated smoke-test script. That let me validate health, search, auth failures, admin authorization, AI validation, SQL-injection-style queries, and XSS-like prompts quickly and repeatedly after each change.

This process was different from my first app because I tested and secured deliberately, not as an afterthought. In App #1, I focused mostly on getting features to appear. For CareCompass, I used clearer requirements, systematic feature testing, responsible AI checks, and security hardening before calling the work complete. The biggest lesson is that production readiness comes from verifying failure paths - invalid forms, unauthorized access, hostile input, and offline errors - as carefully as the happy path.